

CareConnect — Recorded Presentation Script

AI-Powered Patient Follow-Up and Reminder Management System

IIT Roorkee AIOps Capstone | Team 11 | Theme 9 | June 2026

Instructions: Record screen + voice. Estimated duration: 12–15 minutes. Each section covers one or more slides. Speakers can record their sections separately and merge.

SECTION: INTRO

Slide: Slide 1 — Title **Speaker:** Reffel Murzello **Duration:** ~30 seconds

Hello everyone. I'm Reffel Murzello, presenting on behalf of Team 11 from the IIT Roorkee Advanced PG Certification in AI Engineering on Cloud and AIOps program.

Our capstone project is called CareConnect — an AI-powered Patient Follow-Up and Reminder Management System. This is our submission for Theme 9 of the Healthcare domain.

Joining me on this project are my teammates: Manisha Pradhan, Sunit Mukherjee, Vivek Sharma, Pradeep Kumar M, and Mohammed Rohith. Let's get started.

SECTION: TEAM INTRODUCTION

Slide: Slide 2 — Team Members **Speaker:** Manisha Pradhan **Duration:** ~30 seconds

Quick introduction to our team. We split responsibilities across data, development, deployment, and presentation.

I, Manisha Pradhan, led the dataset creation and documentation. Sunit Mukherjee built our RAG pipeline and patient summary agent. Reffel Murzello architected the LangGraph workflow. Vivek Sharma built the Streamlit frontend and the reminder agent. Pradeep Kumar M developed the escalation agent and error handling. And Mohammed Rohith handled our entire AWS deployment.

SECTION: DOMAIN AND PROBLEM

Slide: Slides 3 & 4 — Domain, Theme, Problem Statement **Speaker:** Manisha Pradhan **Duration:** ~90 seconds

We chose Healthcare as our domain — specifically Theme 9, the AI Patient Follow-Up Assistant.

The problem statement from the RFP is clear: hospitals miss follow-up communication. Let me make this concrete. Every year, approximately 20% of hospital patients experience a preventable adverse event within 30 days of discharge — simply because no one followed up.

Care coordinators are manually managing hundreds of patients using spreadsheets. When a patient who just had open-heart surgery is due for follow-up, the current process relies on a

human remembering to call, sending the right message, and escalating if something is wrong. This breaks down constantly. Reminders are generic. High-risk patients aren't reliably identified.

The business impact is staggering — a single preventable readmission costs between fifteen and fifty thousand dollars. Research shows intelligent follow-up systems reduce readmissions by 15 to 25 percent. That's the gap CareConnect fills.

SECTION: PROPOSED SOLUTION

Slide: Slide 5 — Solution Overview **Speaker:** Sunit Mukherjee **Duration:** ~60 seconds

CareConnect is an end-to-end AI system built around three specialized agents, orchestrated by LangGraph.

First, our Patient Summary Agent reads a patient's records and generates a clinical summary along with a risk classification — low, medium, high, or critical.

Second, based on the risk level, our Reminder Agent generates three personalized reminders — an SMS, an email, and a phone call script — all tailored to the patient's specific diagnosis and medications.

Third, for high and critical risk patients, our Escalation Agent generates a structured clinical escalation report with immediate action items, and routes the case to a human clinician for approval before any reminder is sent.

The backbone is GPT-4o for reasoning, ChromaDB for RAG, and the whole thing runs on AWS with a Streamlit dashboard for the care team.

SECTION: DATASET AND DOCUMENTS

Slide: Slide 6 — Dataset **Speaker:** Manisha Pradhan **Duration:** ~60 seconds

We created all our data synthetically — which the RFP explicitly encourages.

Our dataset has three CSV files: 20 patients with realistic medical profiles — diagnoses ranging from acute myocardial infarction and congestive heart failure to breast cancer post-chemotherapy and stroke recovery. We have 40 appointment records and 40 follow-up interaction records.

For the RAG system, we created two clinical text documents. The first — medical followup protocols — is a comprehensive 300-line document covering cardiac, diabetes, oncology, neurological, surgical, and kidney disease follow-up protocols. This is what the AI reads to ground its recommendations.

The second document is a set of realistic patient discharge summaries for our most critical patients, giving the model additional clinical context.

SECTION: ARCHITECTURE

Slide: Slide 7 — System Architecture **Speaker:** Reffel Murzello **Duration:** ~90 seconds

Let me walk you through the architecture.

At the top, we have the Streamlit frontend running on an AWS EC2 instance — accessible on Port 8501.

When the clinician selects a patient and clicks Generate Follow-Up Plan, the frontend calls our LangGraph workflow. The workflow starts by loading the patient's data from our CSV datasets. It then hits the RAG retrieval node — this embeds a diagnosis-specific query and retrieves the most relevant protocol chunks from ChromaDB.

With patient data and protocol context ready, the Patient Summary Agent runs — a GPT-4o call with structured JSON output. It returns a clinical summary and a risk level.

Here's where the Router comes in. If risk is low or medium, we go directly to the Reminder Agent. If it's high or critical, we route to the Escalation Agent, then pause the workflow using LangGraph's built-in interrupt mechanism for human review.

All state is persisted by LangGraph's MemorySaver checkpoint. Documents and datasets are on AWS S3.

SECTION: AGENT WORKFLOW

Slide: Slide 8 — Agent Workflow **Speaker:** Sunit Mukherjee **Duration:** ~60 seconds

Let me go deeper on each agent.

The Patient Summary Agent gets the full patient profile — name, age, diagnosis, medication list, appointment history, follow-up history — plus the RAG context. It calls GPT-4o with a clinical system prompt. The output is structured JSON with risk level, clinical summary, key concerns, recommended actions, and urgency reason.

The Reminder Agent takes the patient data and clinical summary and generates three reminders. The SMS is kept under 160 characters. The email has a warm, patient-friendly body with next steps. The phone script is written for a care coordinator to read — complete with greeting, verification, open-ended questions, and closing instructions.

The Escalation Agent generates a report that includes an escalation ID, a 3-4 sentence clinical brief for the physician, a list of immediate actions, potential complications, and a response timeline.

SECTION: RAG PIPELINE

Slide: Slide 9 — RAG Pipeline **Speaker:** Sunit Mukherjee **Duration:** ~60 seconds

Our RAG pipeline has two phases.

First, the ingestion phase — which runs once at startup. We load text documents from AWS S3 using LangChain's DirectoryLoader. We split them using RecursiveCharacterTextSplitter with a chunk size of 800 characters. Each chunk is embedded using OpenAI's text-embedding-3-small model. These embeddings are stored in ChromaDB, persisted to disk.

Second, the retrieval phase — for every patient. We form a query: 'Follow-up protocol for acute myocardial infarction patient post-discharge.' We embed that query, run a cosine similarity search in ChromaDB, and return the top 4 chunks. These become the rag_context injected into the agents.

The key value of RAG is grounding. Without it, the model might hallucinate clinical guidelines. With our protocol documents, outputs are anchored to clinically accurate information.

SECTION: HUMAN APPROVAL

Slide: Slide 10 — Human-in-the-Loop **Speaker:** Reffel Murzello **Duration:** ~60 seconds

One of the technical highlights of this project is our human-in-the-loop implementation.

LangGraph has a native interrupt() function. When a patient is flagged as high or critical risk, the human_approval_node calls interrupt() with a payload containing the patient ID, clinical summary, and full escalation report.

LangGraph immediately pauses the workflow. The Streamlit frontend detects the interrupt by checking the graph snapshot — specifically, whether 'human_approval' appears in the next nodes list.

The clinician sees a red alert panel with the escalation details and three buttons: Approve, Reject, or Escalate. When the clinician makes a decision, the frontend calls graph.stream with Command(resume=decision) and the same thread ID. LangGraph resumes from the exact paused state.

Approve routes to the Reminder Agent. Reject or Escalate routes to the Output Formatter.

SECTION: AWS DEPLOYMENT

Slide: Slide 11 — AWS Deployment **Speaker:** Mohammed Rohith **Duration:** ~60 seconds

I handled our AWS deployment, so let me walk you through it.

We used a t2.medium EC2 instance running Ubuntu 22.04. This has 2 vCPUs and 4 gigabytes of RAM — sufficient for Streamlit, LangGraph, and ChromaDB to run comfortably.

We set up a security group allowing inbound traffic on Port 8501 for Streamlit and Port 22 for SSH. We assigned an Elastic IP so the public URL stays stable.

For storage, we created an S3 bucket called 'careconnect-documents' with two folders: documents for our protocol text files and datasets for our CSV files. The EC2 instance accesses S3 using IAM role-based credentials — no hardcoded keys.

The RAG pipeline first tries to load documents from S3, falling back to local files if unavailable. The application is launched with streamlit run on port 8501 and is accessible on the public EC2 IP.

SECTION: CHALLENGES

Slide: Slide 12 — Challenges **Speaker:** Pradeep Kumar M **Duration:** ~60 seconds

Let me share the real challenges we faced — because this is where the learning happened.

The biggest challenge was state management. When LangGraph pauses with `interrupt()` and Streamlit re-renders the page, you lose all in-memory state. Our solution was to store the `thread_id` in Streamlit's `session_state` and use `get_state()` to retrieve the full LangGraph snapshot on reload.

Our second challenge was RAG quality. Early on, retrieval was returning irrelevant chunks for specific medical diagnoses. We fixed this by making queries diagnosis-specific and increasing the chunk size to preserve more context per chunk.

We also had issues with GPT-4o sometimes wrapping its JSON responses in markdown code blocks. We fixed this with OpenAI's `response_format` parameter and a `safe_json_parse` utility with regex fallback.

Finally, ChromaDB was using relative paths that broke on EC2. We fixed this by computing all paths dynamically from the absolute path of the config file.

SECTION: KEY OUTCOMES AND DEMO

Slide: Slides 13 & 14 — Results and Demo **Speaker:** Vivek Sharma **Duration:** ~90 seconds

Let me show you what CareConnect looks like in action and what it achieves.

[Begin screen share of Streamlit app]

Here's the CareConnect dashboard. On the left sidebar, we have a dropdown with all 20 patients. Let me select Patient P003 — Suresh Nair, 72 years old, diagnosed with Congestive Heart Failure.

You can see his patient card — it shows a RED CRITICAL badge, his follow-up is overdue, and he's under Dr. Rajan Mehta in Cardiology.

I'll click Generate Follow-Up Plan. Watch the spinner as LangGraph runs...

[Workflow runs — approximately 8-10 seconds]

The system has flagged this as critical. Here's the Human Approval panel. You can see the escalation report — it mentions BNP elevation, fluid overload, and three immediate actions including weight monitoring and urgent cardiology review.

I'll click Approve. The workflow resumes...

Here are the three reminders. The SMS says: 'Reminder: Your follow-up at City Hospital with Dr. Mehta is due. Monitor daily weight. Call 1800-XXX if weight increases 2kg in 2 days.'

The email has a full personalized letter. And here's the phone call script for the coordinator.

In terms of key outcomes: we correctly processed all 20 patients, routed 13 to escalation, and generated 60 total reminders. Follow-up prep time reduced from 45 minutes to under 2 minutes.

SECTION: FUTURE SCOPE

Slide: Slide 15 — Future Scope **Speaker:** Vivek Sharma **Duration:** ~30 seconds

In the short term, we'd integrate with real EMR systems using HL7 FHIR APIs, add actual SMS and email dispatch via Twilio and AWS SES, and support multiple Indian languages for reminders.

Medium term would include a fine-tuned medical LLM, patient mobile app integration, and a predictive readmission risk model using patient history.

Long-term, we envision a multi-hospital deployment with full HIPAA and India's DPDP Act compliance, and voice AI agents that autonomously conduct follow-up calls.

SECTION: CLOSING

Slide: Slide 16 — Thank You **Speaker:** Reffel Murzello **Duration:** ~30 seconds

To summarize: CareConnect is a complete, working, cloud-deployed AI solution for one of healthcare's most persistent operational problems — missed patient follow-up.

We've demonstrated multi-agent workflows, RAG grounding on clinical protocols, human-in-the-loop using LangGraph's interrupt, AWS EC2 and S3 deployment, and a production-quality Streamlit interface.

Thank you for reviewing our project. On behalf of Team 11 — Manisha, Sunit, Vivek, Pradeep, and Mohammed — we appreciate the opportunity to learn and build through this capstone.

This is Team 11 signing off.

Recording Checklist

- Open Streamlit app at localhost:8501 or EC2 public URL
- Open architecture diagram for visual reference
- Open code editor with graph.py visible for the workflow explanation
- Have all slides ready in presentation software
- Test microphone audio quality before recording
- Close unnecessary browser tabs
- Record screen + voice (Loom / OBS / Zoom local recording)
- Keep pace — aim for 12–15 minutes total
- Demonstrate live: patient selection → workflow → approval → reminders
- Export as MP4 (720p or 1080p)
- Upload to Google Drive and get shareable link
- Add link to Recorded_Presentation_Link.txt in submission folder